

Week 1: Requirement Analysis and Medical Shop Management Module

ABINAYA R [E25AI102]

1. Requirement Analysis

Understanding Medical Shop Business

A medical shop management system manages the day-to-day activities of a medical shop. It helps to store and manage medicine details, customer information, supplier details, stock and sales records. The system makes it easier to check medicine availability, update stock, prepare bills and maintain sales information accurately.

The Process Flow:

Medicine Entry	Check Stock	Customer Purchase	Billing / Payment	Stock Update
---------------------------	------------------------	------------------------------	------------------------------	-------------------------

→

Process Flow Description – Medical Shop Management System

- **Medicine Entry:** The staff stores the medicine name, category, price, available quantity and expiry date.
- **Check Stock:** The available quantity of the required medicine is checked before making a sale.
- **Customer Purchase:** The customer selects the required medicine and the staff enters the quantity.
- **Billing / Payment:** The system calculates the total amount and records the payment details.
- **Stock Update:** After the sale, the quantity of the medicine is reduced and the transaction is stored.
- **Supplier Management:** Supplier details are maintained so medicines can be purchased and stock can be replenished.

Entities:

Entity	Description	Key Attributes	Key Relationships
Medicine	Medicine available in the medical shop.	medicine_id (PK), medicine_name, category_id (FK), price, stock_quantity, expiry_date, supplier_id (FK)	Many-to-One with Category and Supplier; One-to-Many with Sale_Item
Category	Groups medicines based on their type.	category_id (PK), category_name	One-to-Many with Medicine
Supplier	Provides medicines to the medical shop.	supplier_id (PK), supplier_name, email, phone, address	One-to-Many with Medicine
Customer	Person who purchases medicines from the shop.	customer_id (PK), customer_name, email, phone, address	One-to-Many with Sale
Sale	Transaction representing a customer's medicine purchase.	sale_id (PK), customer_id (FK), sale_date, total_amount, payment_method	Many-to-One with Customer; One-to-Many with Sale_Item
Sale_Item	Individual medicine included in a sale.	sale_item_id (PK), sale_id (FK), medicine_id	Many-to-One with Sale and Medicine

		(FK), quantity, price, subtotal	
Admin	Manages medicines, suppliers, customers, stock and sales.	admin_id (PK), admin_name, email, password, role	Manages Medicine, Supplier, Customer and Sale

Customer Database Module

The customer database module stores the basic details of customers and connects them with their sales records. This helps the medical shop maintain customer information and refer to previous purchases when required.

Customer Table Creation:

The Customer table is created to store customer details such as customer ID, name, email, phone number, address and registration date.

```
SQL> CREATE TABLE Customer(Customer_ID varchar(5) Primary key, Customer_Name varchar(30) not null, Email varchar(30) not null, phone varchar(10), Address varchar(200), Registration_Date date);
Table created.
```

```
SQL> desc Customer;
+-----+-----+-----+
Name                               Null?   Type
+-----+-----+-----+
CUSTOMER_ID                       NOT NULL VARCHAR2(5)
CUSTOMER_NAME                     NOT NULL VARCHAR2(30)
EMAIL                             NOT NULL VARCHAR2(30)
PHONE                             VARCHAR2(10)
ADDRESS                           VARCHAR2(200)
REGISTRATION_DATE                 DATE
```

Insert Values:

Sample customer records are inserted into the Customer table. Multiple rows can be added to store the details of different customers.

```
SQL> INSERT INTO Customer VALUES ('C006','Anu','anu09@gmail.com','9637460873','Tambaram , Tamilnadu', DATE '2026-05-12');
1 row created.
```

```
SQL> select * from Customer;
```

CUSTO	CUSTOMER_NAME	EMAIL	PHONE
ADDRESS			
REGISTRAT			
C006	Anu	anu09@gmail.com	9637460873
Tambaram , Tamilnadu			
12-MAY-26			
C004	Vijay	vijay07@gmail.com	9635677873
Mannivakam , Tamilnadu			
22-APR-26			

Like this we can create as many rows we want.

Update:

The UPDATE command is used to change an existing customer record. For example, the phone number, address or other customer information can be updated using the customer ID.

```
SQL> update Customer set phone = 9710021922 where customer_ID = 'C009';  
1 row updated.
```

Delete:

The DELETE command is used to remove an unwanted customer record from the table. The record can be deleted by specifying the customer ID.

```
SQL> DELETE FROM Customer WHERE CUSTOMER_ID = 'C008';  
1 row deleted.
```